

BRAINHACK 2026

RoboVerse: Flight Challenge
Intro to Quadcopter Control

AGENDA

1. RoboVerse: Flight Challenge Overview

2. PX4

3. Using Simulator

4. Flight Control using MAVSDK

5. Getting telemetry data

6. Getting camera video stream

RoboVerse: Flight Challenge Overview

RoboVerse: Flight Challenge Qualifier

- RoboVerse: Flight Challenge is a drone autonomy challenge.
- The challenge requires teams to come up autonomous solutions to solve drone challenges in a two stages competition - Qualifier and Final.
- In the Qualifier round, the teams are to navigate a virtual drone to search for fuel barrels and map a virtual space depot.
- The Qualifier round takes place on 22 and 23 May 2026 physically at Grand Court Hotel Orchard.
- Top 26 teams will be invited to the Final held on 10-11 June 2026 at Marina Bay Sands (Expo and Convention Centre)



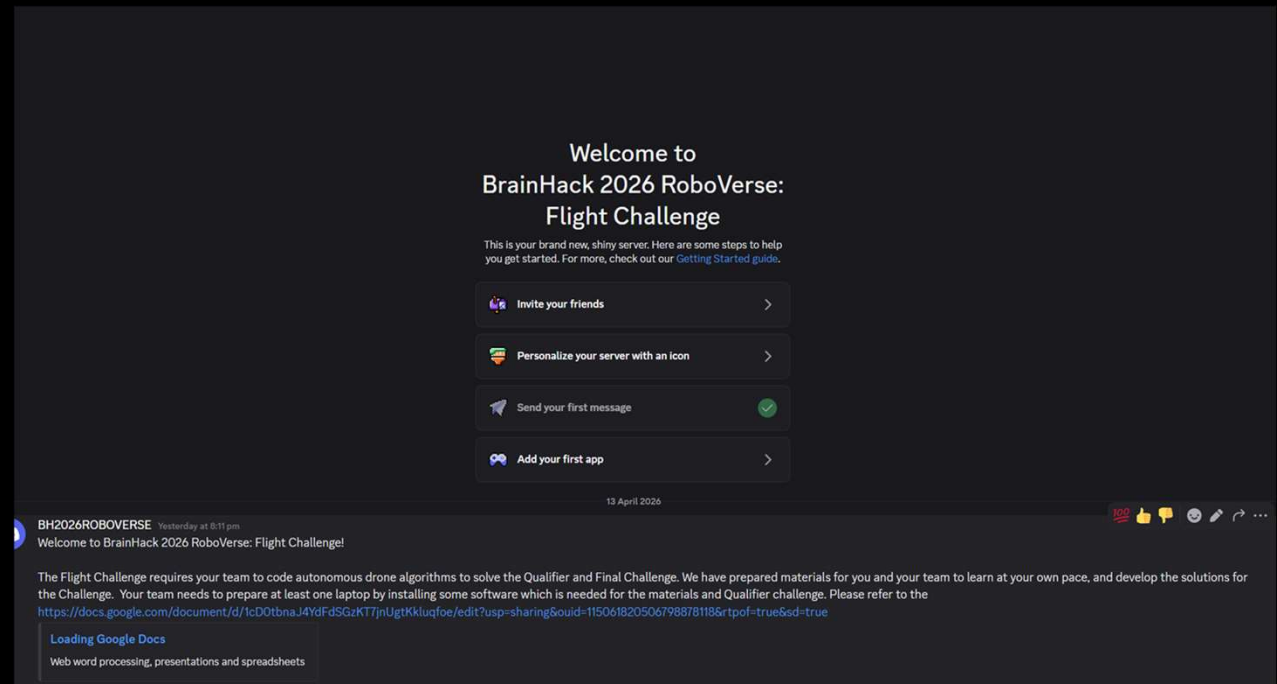
RoboVerse: Flight Challenge Final

- In the final round, teams are tasked with mapping an unknown area using a physical drone equipped with a depth camera, followed by coordinating a swarm of drones to locate targets.



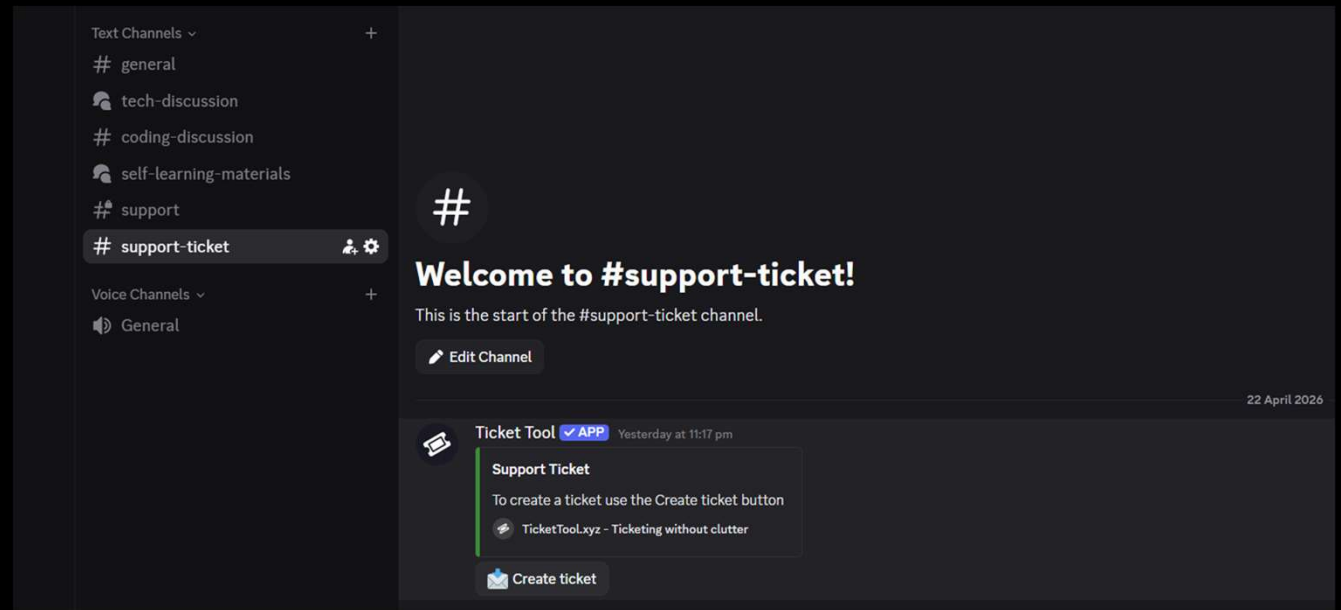
Journey to Final

- Learning packs are on RoboVerse: Flight Challenge Discord server, for teams to learn on their own pace.
- Teams can seek consultations from the Coordinators via the Discord server.
- Invitation link will be sent out. Need to provide email address for verification.
- Once onboard, you raise a ticket to get support from the Coordinators.



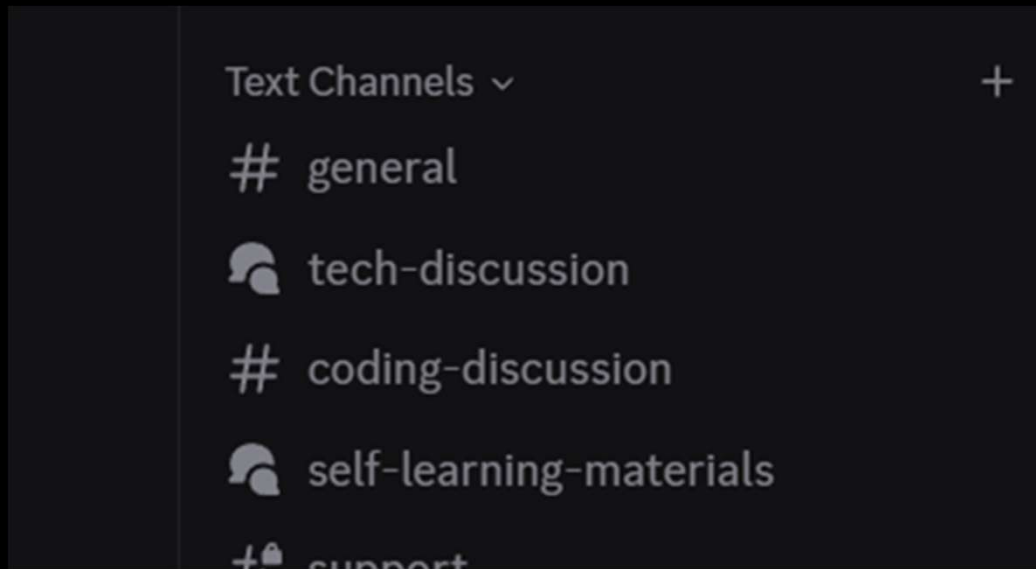
Technical Help

- Teams can ask Coordinators questions via #support-ticket.
- Click on # support-ticket and then Create ticket. Then describe the question.
- One question per ticket.
- Our Coordinators will follow-up closely on the ticket.
- Once the Coordinator has addressed your question, you may close the ticket.
- Ticket is private and thus other team cannot see.



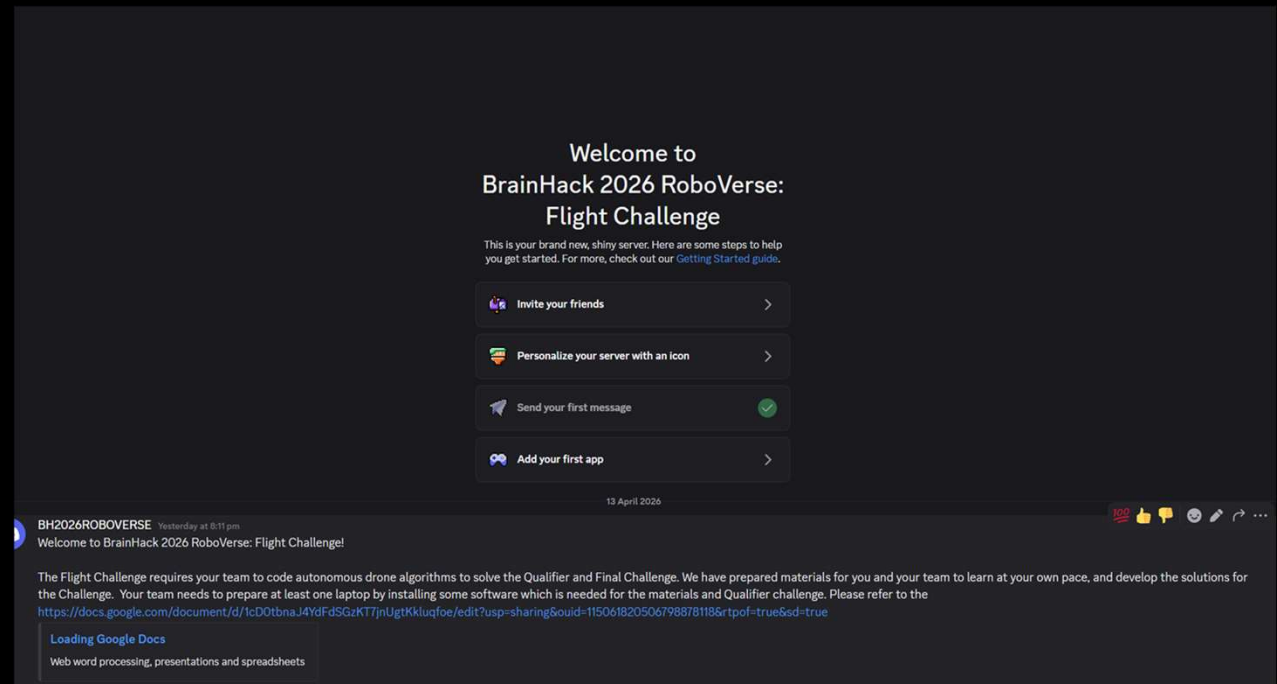
Discussion Channels

- There are topic specific discussion channel.
- These are for all teams to share and discuss on learning.
- All teams can see.
- Our Coordinators may chip in and thus may or may not follow up on questions raised on discussion channel. They will definitely follow up on question raised via #support-ticket.



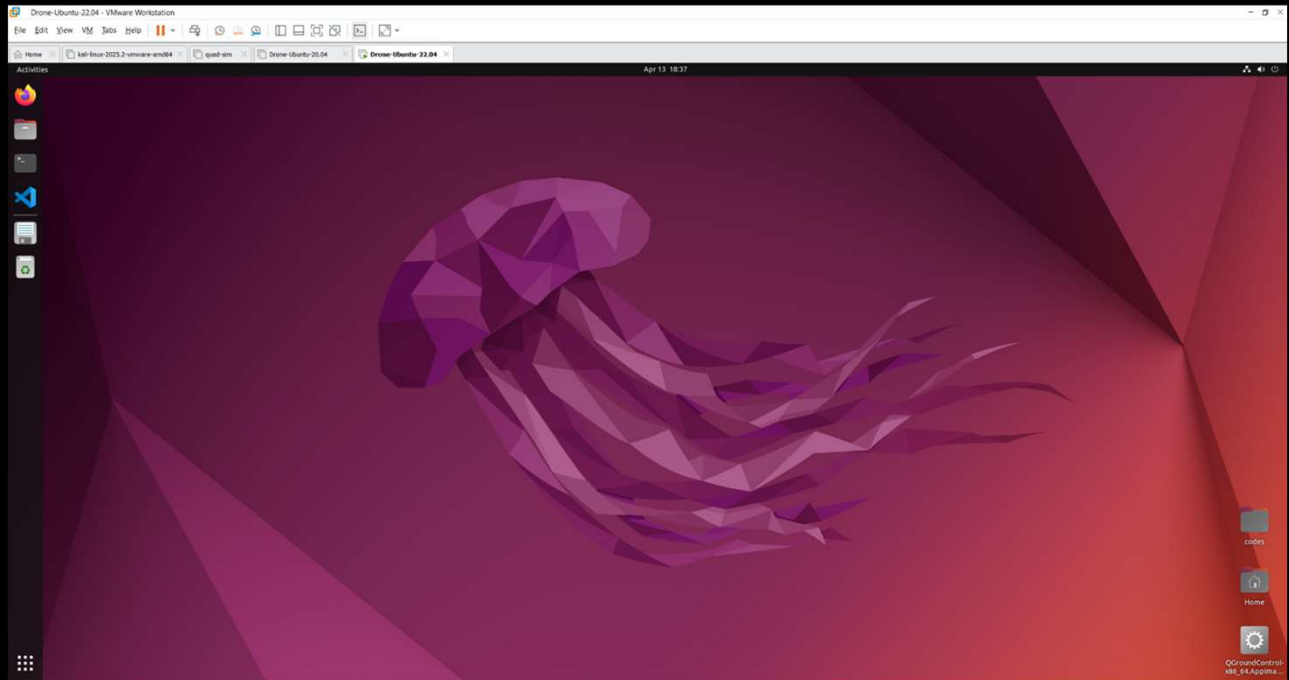
5 Learning Packs

- The Learning Packs consist of videos, sample codes, and theoretical resources covering potential approaches to the challenges.
- Three (3) main Learning Packs cover quadcopter control, obstacle avoidance and mapping, and object detection.
- Two (2) supplementary Learning Packs for the Final focus on visual odometry.
- Coordinators are on RoboVerse Discord server to provide guidance and technical assistance.



Platform used for Qualifier and Final

- PX4-based quadcopter
- Development environment:
 - Ubuntu 22.04 (with development essentials c++, python), QGroundControl and MAVSDK-python and MAVSDK-c++
- A VMWare virtual desktop machine that has the development platform installed in it will be provided.
 - VMWare machine installation guide and link to download the virtual machine are on Discord server.
- Teams can also prepare their own development environment.
- Teams can seek advice from Coordinators on the Discord server on installations and usage.

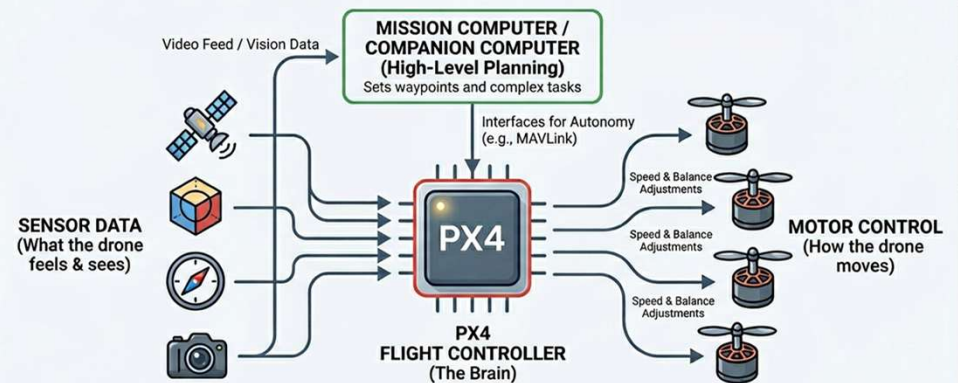


PX4

What is PX4?

- PX4 is a quadcopter autopilot platform used in both the Qualifier and Final challenges.
- The autopilot acts as the brain of the quadcopter, processing sensor data and controlling the motors to achieve stable flight.
- The flight stack provides essential stabilization and safety features, as well as a level of pilot assistance for both manual and autonomous flight.

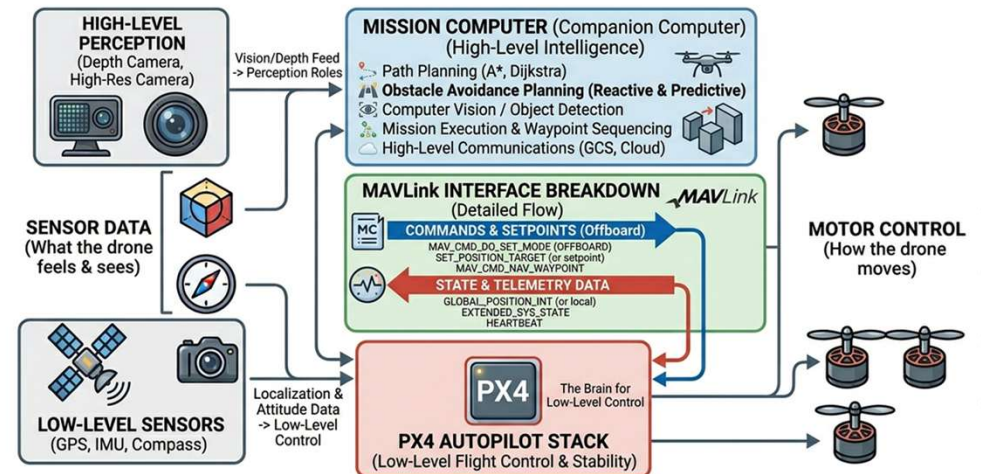
PX4: The Flight Control Brain of a Quadcopter



Mission Computer

- Mission Computer or companion computer performs high-level intelligence and then control the PX4.
- The PX4 passes the control “offboard” to the Mission Computer.
- The PX4 and Mission Computer communicate over a high-speed serial or IP link using the MAVLink protocol.
- Most Mission Computers run a Linux-based OS such as Ubuntu 22.04.
- The challenges require teams to develop the control logic that runs on the Mission Computer.
- During the Qualifier, the Mission Computer will be your code running on either the team’s laptop or the organiser’s laptop.

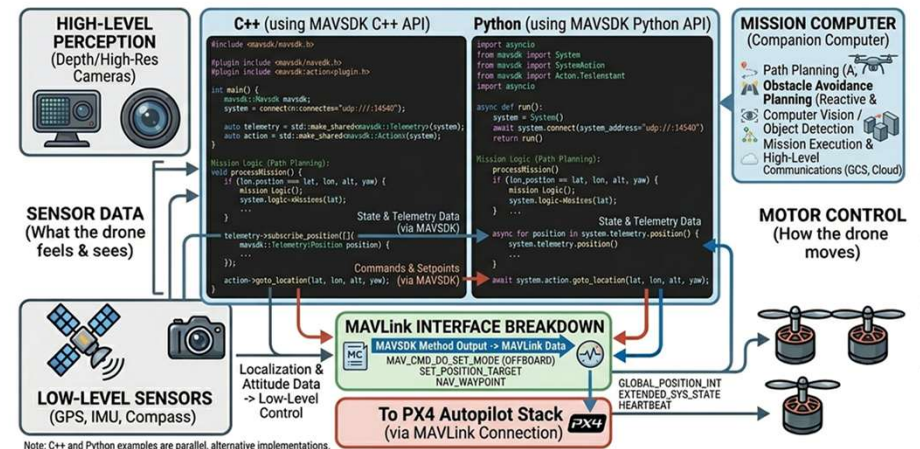
Detailed Interface & Roles: Mission Computer with PX4 Autopilot



MAVSDK

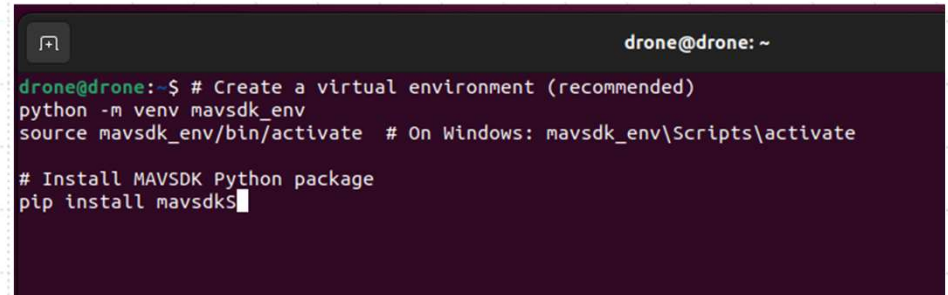
- MAVLink (Micro Air Vehicle Link) is a lightweight, header-only messaging protocol designed for bandwidth-constrained radio links to communicate between unmanned vehicles (drones) and ground control stations (GCS) or internal components.
- MAVSDK (or MAV Software Development Kit) is a set of high-level, cross-platform libraries that take cares of sending MAVLink-instructions to MAVLink supported drones and systems (like PX4).
- MAVSDK supports languages including C++, Python, Swift, and Java.
- Your team mainly focuses on developing the code in the mission computer for the high level autonomy logic.
- The workshop focuses mostly on python and sometimes C++.

MISSION COMPUTER INTERNAL LOGIC: MAVSDK INTERFACE to PX4



Install MAVSDK python

- On the terminal console of the Mission Computer, install MAVSDK by entering the following commands.
- The virtual machine provided has MAVSDK and no further installation is needed.

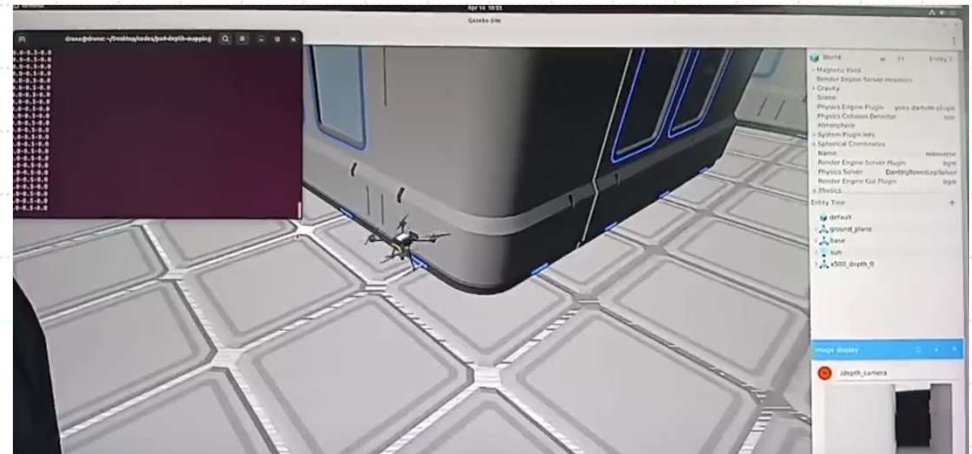
A terminal window with a dark background and light text. The title bar shows 'drone@drone: ~'. The terminal content shows a series of commands for creating a virtual environment and installing MAVSDK. The commands are: 'python -m venv mavsdk_env', 'source mavsdk_env/bin/activate' (with a note for Windows), and 'pip install mavsdk'.

```
drone@drone: ~  
drone@drone:~$ # Create a virtual environment (recommended)  
python -m venv mavsdk_env  
source mavsdk_env/bin/activate # On Windows: mavsdk_env\Scripts\activate  
  
# Install MAVSDK Python package  
pip install mavsdk
```

Using Simulator

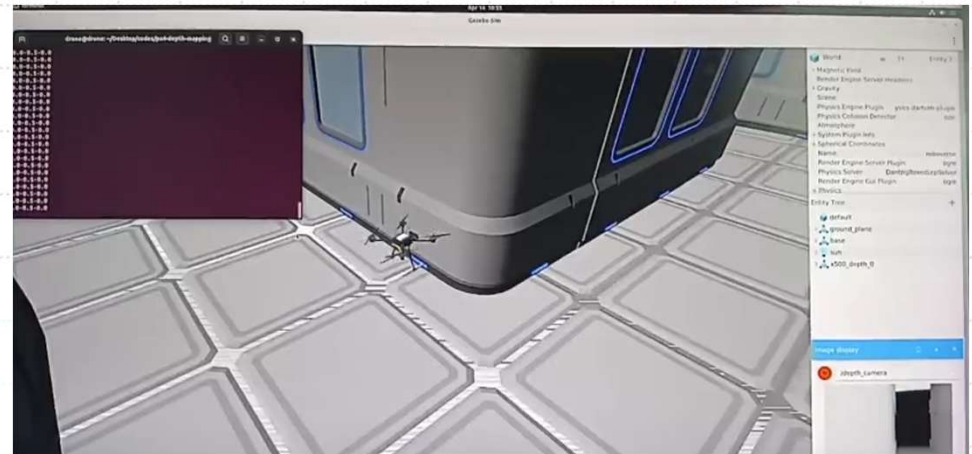
Simulator

- The Qualifier challenge and workshop uses simulation environment. “World” is simulated using Gazebo Harmonic.
- The simulation environment is in the VMWare virtual desktop machine, and runs within the virtual machine.
- PX4 also runs within the virtual machine in Software-In-The-Loop mode (SITL PX4), to emulate a real-life drone in the simulation environment.
- The Gazebo gives SITL PX4 the simulated sensors data, and also provides camera or depth camera feeds to your code.
- Your code sends commands to PX4 SITL, which then controls the virtual drone in the simulation.
- Ground Control Station, QGroundControl, monitors and controls the emulated PX4.
 - QGroundControl has been installed into the VMWare virtual desktop machine.



PX4 Software In The Loop (SITL)

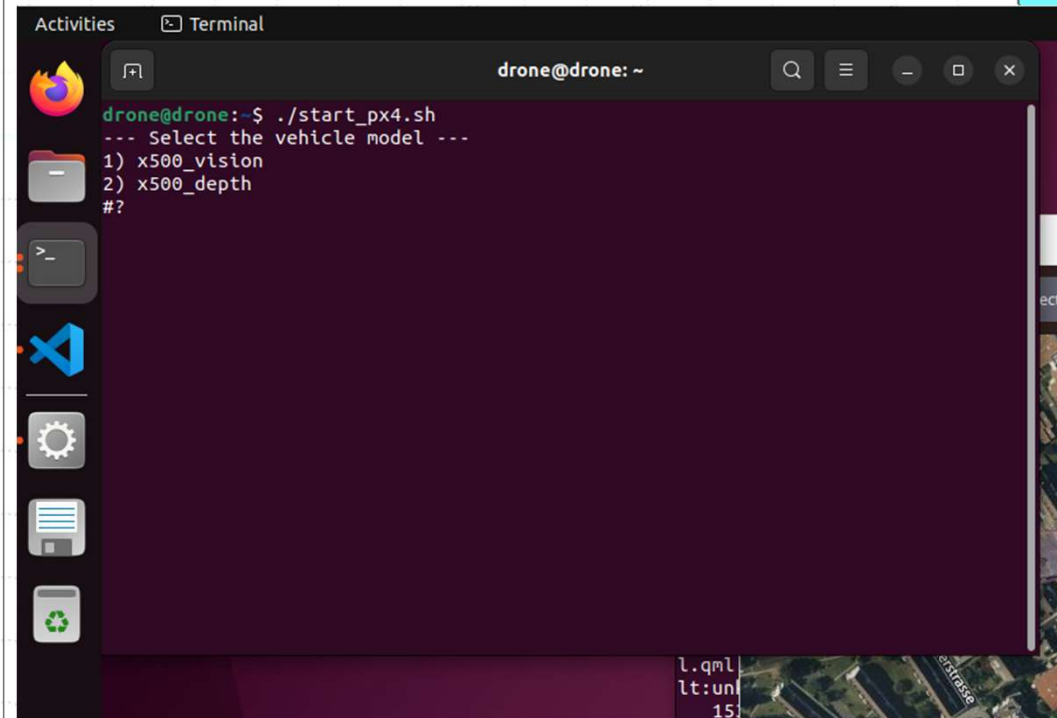
- A copy of the PX4 source code needs to be replicated onto your machine.
- The provided VMWare virtual machine has a copy of PX4 source code.
- The PX4 SITL has virtual drones that have cameras, depth cameras and camera with gimbal.
- The drone that we are going to use is gz_x500_vision or gz_x500_depth.
- To run the virtual drone, need to “compile” the source code of the PX4 which will then start the virtual environment.
- Once spawned, it will keep running until you CTRL+C it.
- Once spawned, you can run your codes.



[INSTRUCTOR TO DEMO]

Steps to Start/Build Simulation

- Open a new Terminal Window and type the following command:
 - **cd ~**
 - **./start_px4.sh**
- Then choose the vehicle by entering 1 or 2
- Then choose the simulation world.
- Lastly, wish to start QGroundControl.



The screenshot shows a terminal window titled "Terminal" with the prompt "drone@drone: ~". The user has executed the command `./start_px4.sh`. The script displays the following text:

```
drone@drone:~$ ./start_px4.sh
--- Select the vehicle model ---
1) x500_vision
2) x500_depth
#?
```

The terminal window is part of a desktop environment with a sidebar on the left containing icons for Activities, Home, Files, Terminal, Visual Studio Code, Settings, and a Recycle Bin. The bottom of the screen shows a taskbar with a clock displaying 15:15 and a small map preview.

Flight Control using MAVSDK

First Code: Takeoff and Land

- Refer to `takeoff_and_land.py`.
- Key concepts in the code:
- When PX4 drone is turned on, it broadcasts a heartbeat MAVLink via udp port 14540 by default. This code make your program listens at port 14540:
`drone.connect(system_address="udpin://0.0.0.0:14540")`
- Once detect a heartbeat, it connects to the drone that advertise the heartbeat.
- All function are called using `await` which means pause the execution of a coroutine until a specific asynchronous task (an "awaitable") is complete because MAVSDK python is asynchronous.
- Uses `await asyncio.sleep(5)` instead of `time.sleep`.
- Wait for successful connection before proceeding.
- Check for successful landing.

[INSTRUCTOR TO DEMO AND EXPLAIN CODE]

Considerations

- Graceful Exit: Pressing `Ctrl+C` will raise `KeyboardInterrupt`. In production code, wrap `asyncio.run(run())` in a try/except or use signal handlers to send a safe `drone.action.land()` or `drone.action.kill()` before exiting.
- SITL vs Real Hardware: This code works identically on a real drone, but you must change `system_address` to `"serial:///dev/ttyACM0:57600"` if the your code runs on single board computer that connects to the PX4 flight controller via serial port or UDP/TCP address if the drone has remote access wifi.

Refer to basic_offboard_control.py

- Autonomy capabilities are achieved using companion computer that performs autonomous logic, instead of the PX4 flight control.
- Then the companion computer then control PX4 via offboard mode. PX4 handles the low-level motor controls based on the high level instructions from companion computer.
- Offboard mode means that the flight controller relinquish autonomous flight logics to an external computer via MAVLINK. Key concepts when using offboard mode:
- Start by calling `offboard.start()` and end use `offboard.stop()`.
- PX4 requires offboard setpoints at least every 0.5 seconds. Meaning your code must send a setpoint command every 0.5 seconds (notice the `asyncio.sleep(0.2)`). If not, PX4 will switch back control to the next flight mode as a form of failsafe.
- The code demonstrates controlling the PX4 using set velocity setpoint command - `await drone.offboard.set_velocity_ned(vel)`
- Important concept for velocity: The Coordinate frame of PX4 is North (+X), East (+y) and Down (+z) Yaw is clockwise from north. -ve for z means up.

[INSTRUCTOR TO DEMO AND EXPLAIN CODE]

Important Note for Offboard control

- Never use `time.sleep()`: It blocks the asyncio event loop, causing MAVLink heartbeats to drop and PX4 to trigger failsafes.
- SITL vs Real Drone: This code is identical for hardware. Only change `system_address` and ensure:
- Radio/Telemetry link is stable
- Offboard mode is permitted in your PX4 parameters (`COM_RC_IN_MODE=1`, `COM_OBL_ACT=2`, etc.)
- You have a reliable way to trigger RC override or failsafe if needed
- Real Drone: Mode sequences using RC - ARM -> POSITION->OFFBOARD
- Debugging Tip: Add telemetry monitoring during offboard:

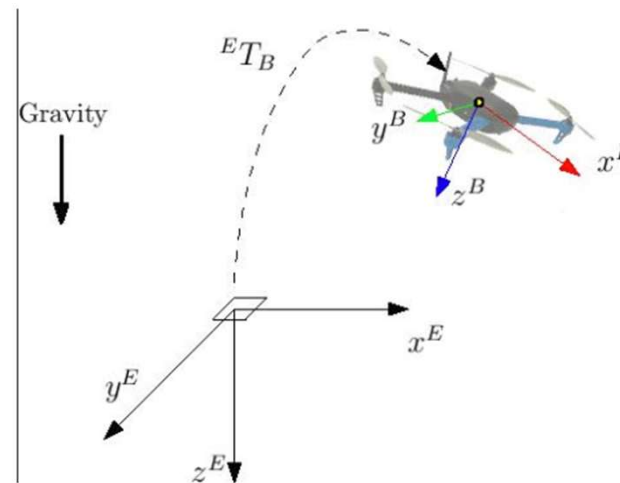
```
1 async for pos in drone.telemetry.position():
2     print(f"Pos: {pos.latitude_deg:.6f}, {pos.longitude_deg:.6f}, {pos.absolute_altitude_m:.2f}m")
3     break # Just sample once if you don't want console spam
```


Coordinates Frames of PX4

- Take note especially when you are using using API that is based on body frame or world frame. Body frame means with respect of the body of the quadcopter.
- Refer to velocity_body.py for controlling velocity.

Frame	PX4
Body	FRD (X Forward, Y Right, Z Down)
World	FRD or NED (X North, Y East, Z Down)

[INSTRUCTOR TO DEMO AND EXPLAIN
VELOCITY_BODY.PY]



Key concepts in Fly_to_coordinate.py

- `drone.action.goto_location(lat, lon, alt, yaw)`: High-level PX4 command. Automatically switches to HOLD → GOTO mode and uses PX4's internal position controller.
- NED vs GPS: This example uses Global Coordinates (Lat/Lon/Alt MSL). PX4 internally converts to NED relative to home.
- Arrival Detection: MAVSDK doesn't provide a native "arrived" callback. You must poll `drone.telemetry.position()` and compute distance. The Haversine formula handles Earth curvature accurately for drone-scale distances.
- Tolerance & Timeout: Real systems oscillate. `pos_tolerance=1.0m` and `alt_tolerance=0.5m` prevent infinite waiting. Timeout prevents script hang on PX4 navigation failures.
- Real Hardware Warning: On physical drones, always: Test in `COM_RC_IN_MODE=1` (Offboard/MAVLink allowed)
- Keep RC sticks ready for manual override, especially switch out of offboard mode into hold mode when something went wrong.
- Set `COM_OBL_ACT=1` (Land) or 2 (Hold) for offboard failures

[INSTRUCTOR TO DEMO AND EXPLAIN FLY_TO_COORDINATE.PY]

```
68 async def run():
69     drone = System()
70     print("✋ Connecting to PX4 SITL...")
71     await drone.connect(system_address="udpin://0.0.0.0:14540")
72
73     try:
74         await explicit_system_checks(drone)
75
76         print("-- Arming")
77         await drone.action.arm()
78
79         print("🚀 Taking off...")
80         await drone.action.set_takeoff_altitude(5.0)
81         await drone.action.takeoff()
82         await asyncio.sleep(8)
83
84         # Target coordinates
85
86         TARGET_LAT, TARGET_LON, TARGET_ALT = 100, 8.545594, 8.0
87
88         print(f"📍 Commanding goto...")
89         # Note: goto_location usually takes (lat, lon, alt_msl, yaw)
90         await drone.action.goto_location(TARGET_LAT, TARGET_LON, TARGET_ALT, 0.0)
91
92         await wait_for_arrival(drone, TARGET_LAT, TARGET_LON, TARGET_ALT)
93
94         print("🛑 Holding for 3 seconds...")
95         await asyncio.sleep(3)
96
```

Getting Telemetry Data

Getting IMU data for state estimation and control feedback

- The python code, imu.py, connects to PX4 using udp, and ask streams of Inertia Measurement Unit (IMU) at a rate of 200Hz using `telemetry.imu()`.
- Prints sensor readings: acceleration, angular velocity, magnetic field and temperature.
- Take note of `_frd` which means the data is in coordinate frame of (Forward-Right-Down)
- IMU are often come in handy by complementing position estimation:
 - Fast angular rate for attitude stablization
 - Acceleration data for feedforward control

[INSTRUCTOR TO DEMO AND then EXPLAIN IMU.PY]

```
8  async def get_imu_data():
9      # Connect to the drone
10     drone = System()
11     await drone.connect(system_address="udpin://0.0.0.0:14540")
12
13     # Wait for the drone to connect
14     print("Waiting for drone to connect...")
15     async for state in drone.core.connection_state():
16         if state.is_connected:
17             print("Drone is connected!")
18             break
19
20     telemetry = drone.telemetry
21
22     # Set the rate at which IMU data is updated (in Hz)
23     await telemetry.set_rate_imu(200.0)
24
25     # Fetch and print IMU data
26     print("Fetching IMU data...")
27     while run:
28         async for imu in telemetry.imu():
29             # Print data in HIGHRES IMU format
30             print("HIGHRES_IMU (105)")
31             print(f"Time (us): {imu.timestamp_us}")
32             print(f"X Acceleration (m/s^2): {imu.acceleration_frd.forward_m_s2}")
33             print(f"Y Acceleration (m/s^2): {imu.acceleration_frd.right_m_s2}")
34             print(f"Z Acceleration (m/s^2): {imu.acceleration_frd.down_m_s2}")
35             print(f"X Gyro (rad/s): {imu.angular_velocity_frd.forward_rad_s}")
36             print(f"Y Gyro (rad/s): {imu.angular_velocity_frd.right_rad_s}")
37             print(f"Z Gyro (rad/s): {imu.angular_velocity_frd.down_rad_s}")
38             print(f"X Mag (gauss): {imu.magnetic_field_frd.forward_gauss}")
39             print(f"Y Mag (gauss): {imu.magnetic_field_frd.right_gauss}")
40             print(f"Z Mag (gauss): {imu.magnetic_field_frd.down_gauss}")
41             print(f"Temperature (°C): {imu.temperature_degC}")
42             print("-----")
43     print("program ends")
```


Get other data like battery, and flight mode

- `get_battery.py` shows how to get battery level using MAVSDK async function which requires your code to provide a callback function to MAVSDK to call.
- `get_flightmode.py` shows how to use MAVSDK to find out flight mode of the PX4.
- `is_arm_air.py` shows how to check if drone is in the air.

[INSTRUCTOR TO DEMO AND then EXPLAIN `get_battery.PY`]

```
30 async def print_battery(drone):
31     async for battery in drone.telemetry.battery():
32         print(f"Battery: {battery.remaining_percent}")
33
34
35 async def print_gps_info(drone):
36     async for gps_info in drone.telemetry.gps_info():
37         print(f"GPS info: {gps_info}")
38
39
40 async def print_in_air(drone):
41     async for in_air in drone.telemetry.in_air():
42         print(f"In air: {in_air}")
43
44
45 async def print_position(drone):
46     async for position in drone.telemetry.position():
47         print(position)
48
```


Getting camera video stream

Get video feed using OpenCV and GZ-transport

- The MAVSDK does not provide means to get video stream.
- Gazebo provides video stream using gz-transport where it provides frame by frame.
- GZ-Transport uses subscribe-publish mechanism where it will publish data to all subscribers whenever new data is available.
- Therefore, just need subscribe to topic and wait for call from gz-transport.
- Refer to get_video.py to see how to subscribe, wait and then display image on the screen.
- When doing object detection, your code might have to do it more efficiently to prevent object detection from holding up your program. We will study that in subsequent workshop.

[INSTRUCTOR TO DEMO AND then EXPLAIN get_video.py]

```
1 import time
2 import numpy as np
3 import cv2
4 # Import Gazebo transport and message bindings
5 from gz.transport13 import Node
6 from gz.msgs10.image_pb2 import Image
7
8 def image_callback(msg: Image):
9     """
10     Callback function that triggers every time a new image message is received.
11     """
12     # Gazebo images are sent as raw bytes. Convert them to a NumPy array.
13     # Typical Gazebo camera encoding is 'rgb_8' (24-bit RGB)
14     frame = np.frombuffer(msg.data, dtype=np.uint8)
15
16     # Reshape the flat array into (height, width, channels)
17     # Note: msg.step is the number of bytes per row
18     frame = frame.reshape((msg.height, msg.width, 3))
19
20     # Convert RGB (Gazebo default) to BGR (OpenCV default) for correct colors
21     frame_bgr = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
22
23     # Display the resulting frame
24     cv2.imshow("Gazebo Live Feed", frame_bgr)
25     cv2.waitKey(1)
26
27 def main():
28     # 1. Initialize the Gazebo Transport Node
29     node = Node()
30
31     # 2. Define the topic name (use 'gz topic -l' to find yours)
32     # Common PX4 camera topic is often /camera or /world/<world_name>/model/<model_name>/link/link/sensor/camera/image
33     image_topic = "/world/roboverse/model/x500_mono_cam_0/link/camera_link/sensor/camera/image"
34
35     # 3. Subscribe to the topic
36     if node.subscribe(Image, image_topic, image_callback):
37         print(f"Subscribed to {image_topic}. Press Ctrl+C to stop.")
38     else:
39         print(f"Failed to subscribe to {image_topic}. Is Gazebo running?")
40         return
41
42     # 4. Keep the script alive to process incoming messages
43     try:
44         while True:
45             time.sleep(0.01)
46     except KeyboardInterrupt:
47         print("Shutting down...")
48     finally:
49         cv2.destroyAllWindows()
50
51 if __name__ == "__main__":
52     main()
53
```

Summary

- This workshop provides the base building blocks for developing autonomy codes in the mission computer using MAVSDK.
- Often, the autonomy codes process telemetry data from drone and camera data, to determine trajectory and then setpoints to command the movement of the quadcopters. Then again, get data as form of closed-loop feedback to the setpoints.

References or Additional Materials

- https://docs.px4.io/main/en/getting_started/px4_basics_concepts
- MAVSDK python sample codes:
<https://github.com/mavlink/MAVSDK-Python/tree/main/examples>
- MAVSDK Python Action API Documentation:
<http://mavsdk-python-docs.s3-website.eu-central-1.amazonaws.com/plugins/action.html>
- MAVSDK Python System API Documentation:
<http://mavsdk-python-docs.s3-website.eu-central-1.amazonaws.com/system.html#mavsdk.system.System>
- PX4 Coordinate frame:
https://docs.px4.io/main/en/ros2/user_guide

THANK YOU